



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 3 – Polymorphism and Virtual Function
Topic – 3.3 Virtual Destructor



Virtual Destructor

- A **destructor in C++** is a member function of a class used to free the space occupied by an object of the class that goes out of scope.
- A destructor has the same name as the name of the constructor function in a class, but the destructor uses a tilde (**~**) sign before its name.

Virtual Destructor:

- A virtual destructor is used to free up the memory space allocated by the derived class object while deleting instances of the derived class using a base class pointer object.
- A base class destructor use the **virtual** keyword that ensures both base class and the derived class destructor will be called at run time.



Why we use virtual destructor in C++?

- When an object in the class goes out of scope or the execution of the main() function is about to end, a destructor is automatically called into the program to free up the space occupied by the object.
- When a pointer object of the base class is deleted that points to the derived class, only the base class destructor is called due to the early bind by the compiler.
- In this way, it skips calling the derived class' destructor, which leads to memory leaks issue in the program.
- And when we use virtual keyword preceded by the destructor tilde (~) sign inside the base class, it guarantees that first the derived class' destructor is called then the base class' destructor is called to release the space occupied by both destructors.



A program to display a class destructor's undefined behaviour without using a virtual destructor in C++

```
class Base
{
public:
    Base()
    {
        cout<< "\n Constructor Base class";
    }
    ~Base()
    {
        cout<< "\n Destructor Base class";
    }
};

class Derived: public Base
{
public:
    Derived()
    {
        cout<< "\n Constructor Derived class";
    }
}
```

```
~Derived()
{
    cout << "\n Destructor Derived class" ;
}
};

int main()
{
    Base *bptr = new Derived;
    delete bptr;
}

Output:
```

```
C:\Users\AMIT YADAV\Documents\dest.exe

Constructor Base class
Constructor Derived class
Destructor Base class
-----
Process exited after 0.03122 seconds with return value 0
Press any key to continue . . .
```

- As we can see in the output, when the compiler compiles the code, it calls a pointer object in the main function that refers to the base class.
- So, it executes the base class' constructor() function and then moves to the derived class' constructor() function.
- Then base class pointer only removes the base class's destructor without calling the derived class' destructor in the program.
- Hence, it leaks the memory in the program.



A program to display a class destructor's behavior using a virtual destructor in C++

```
class Base
{
public:
    Base()
    {
        cout << "\n Constructor Base class";
    }
    virtual ~Base()      //virtual destructor
    {
        cout << "\n Destructor Base class";
    }
};

class Derived: public Base
{
public:
    Derived()
    {
        cout << "\n Constructor Derived class" ;
    }
}
```

```
~Derived()
{
    cout << "\n Destructor Derived class";
}

int main()
{
    Base *bptr = new Derived; // A pointer object reference
    delete bptr;
}

Output:
```

```
C:\Users\AMIT YADAV\Documents\dest.exe

Constructor Base class
Constructor Derived class
Destructor Derived class
Destructor Base class
-----
Process exited after 0.06323 seconds
Press any key to continue . . .
```



In the previous program, we have used a virtual destructor inside the base class that calls the derived class' destructor before calling the base class' destructor and release the space or resolve the memory leak issues in the program.

Thank You..